HOUSE VALUE PREDICTION USING LINEAR REGRESSION

A Project Report

Submitted by:

Vikas Kumar 25MDA10098

in partial fulfillment for the award of the degree of

MASTER OF BUSINESS ADMINISTRATION

IN

(DATA SCIENCE AND ARTIFICIAL INTELLIGENCE)

Chandigarh University

Academic Year 2026–2027

CHAPTER 1

INTRODUCTION

1.1 Background and Motivation

The value of a house or residential area depends on several factors, including location, household income, the age of buildings, the number of rooms and population characteristics. Estimating values manually across thousands of areas can be time-consuming. Machine learning can identify patterns in historical data and use those patterns to estimate values for new records.

Linear Regression is a supervised machine-learning algorithm used to predict a numerical value. It estimates a relationship between input features and a target using a weighted linear equation. It is widely used as a baseline because it is relatively simple, fast to train and easy to interpret.

1.2 Problem Statement

This project uses eight census-based characteristics of a California district to predict its median house value. Since the output is numerical, the task is a regression problem. The objective is to prepare the dataset, train a Linear Regression model and evaluate its predictions on records that were not used during training.

1.3 Project Objectives

- Load a real, documented housing dataset rather than generated sample data.
- Explore the data using summary statistics and visualisations.
- Separate input features from the target variable and split the data into training and testing sets.
- Standardise features within a pipeline to support reproducible processing and prevent test-data leakage.
- Train a Linear Regression model for house value prediction.
- Evaluate model performance using MAE, RMSE and R^2.
- Save evaluation metrics and actual-versus-predicted results.

1.4 Scope of the Project

The project covers structured district-level data, data preprocessing, one regression model, evaluation and saved output files. The dataset describes California districts from the 1990 United States census. It predicts the median value of a district rather than the price of an individual house. It is an academic prototype and must not be used to estimate current house prices in India or other regions.

CHAPTER 2

DATASET AND METHODOLOGY

2.1 Dataset Source and Description

The project uses the California Housing dataset available through scikit-learn with the function fetch_california_housing. The dataset is based on the 1990 United States census. Each row describes a census block group, a small geographic area. The dataset is downloaded automatically the first time the script runs, so an internet connection is required for the initial download.

Characteristic	**Description**
Number of records	20,640
Number of input features	8
Target variable	MedHouseVal
Target unit	$100,000 (1.0 = $100,000)
Task	Regression
Data type	Structured tabular data
Missing values	None reported in the source report

2.2 Features and Target Variable

All eight input columns are used to describe district income, housing, population and location. The target variable is the median house value of the district.

Feature	**Meaning**
MedInc	Median household income in units of $10,000
HouseAge	Median age of houses in years
AveRooms	Average number of rooms per household
AveBedrms	Average number of bedrooms per household
Population	Number of people living in the district
AveOccup	Average household occupancy
Latitude	North-south geographic position
Longitude	East-west geographic position
MedHouseVal (target)	Median house value in units of $100,000

2.3 Exploratory Data Analysis

EDA observation	**Finding**
Number of records	20,640

Lowest median house value	Approximately $15,000
Median house value	Approximately $179,700
Average house value	Approximately $206,900
Highest recorded median value	$500,000 (data cap)
Standard deviation	Approximately $115,400
Districts at the $500,000 cap	992 (about 4.8%)
Feature most related to value	MedInc (correlation approximately 0.69)

The dataset contains a range of house values, with a group of expensive districts recorded at the maximum value of $500,000. This is a cap in the original dataset, not evidence that all these districts had identical true values. Median household income has the strongest reported single-feature correlation with house value. Location also matters, with expensive districts clustered around coastal areas such as the San Francisco Bay Area and Los Angeles.

Some features contain extreme values, including average rooms per household and average occupancy. These values are retained because they are part of the original dataset. They should be considered when interpreting the model's performance.

2.4 Feature Scaling

The input features use different measurement scales. StandardScaler transforms each feature to have a mean close to zero and a standard deviation close to one. It is placed inside a scikit-learn Pipeline with Linear Regression. The scaler is fitted on the training set only and then applied to the test set, helping prevent information leakage. Scaling is not required for ordinary least-squares Linear Regression predictions, but it makes the preprocessing steps explicit and helps interpret standardised coefficients.

2.5 Train-Test Split

Subset	**Records**	**Purpose**
Training set	16,512	Fit the model parameters
Testing set	4,128	Evaluate predictions on unseen records
Total	20,640	Complete dataset

The dataset is split into training and testing sets in an 80:20 ratio using random_state=42. The test set is not used to fit the model and is reserved for evaluation. A fixed random state makes the split reproducible.

2.6 Linear Regression Model

Linear Regression estimates the target as a weighted sum of input features. Each feature receives a coefficient, representing its relationship with the predicted value while the other features are held constant. The model is easy to train and interpret. However, it may not capture complex non-linear patterns or interactions between features.

2.7 Evaluation Metrics

2.7.1 Mean Absolute Error (MAE)

MAE is the average absolute difference between actual and predicted values. It is easy to interpret and lower values indicate smaller average errors. Because the target is in units of $100,000, an MAE of 0.534 is equivalent to approximately $53,400.

$$MAE = (1/n) \Sigma |y_i - \hat{y}_i|$$

2.7.2 Root Mean Squared Error (RMSE)

RMSE is the square root of the average squared prediction error. Large errors affect RMSE more strongly than small errors. Lower values indicate better performance.

$$RMSE = \sqrt{[(1/n) \Sigma (y_i - \hat{y}_i)^2]}$$

2.7.3 Coefficient of Determination (R^2)

R^2 measures how much of the variation in the target is explained by the model compared with predicting the mean. A value closer to 1 indicates a stronger fit.

$$R^2 = 1 - [\Sigma(y_i - \hat{y}_i)^2 / \Sigma(y_i - \bar{y})^2]$$

CHAPTER 3

IMPLEMENTATION

3.1 Tools and Technologies

Technology	Purpose
Python	Main programming language
Pandas	Data handling and saving result tables
Matplotlib	Creating plots and visualising results
Scikit-learn	Dataset loading, preprocessing, Linear Regression and metrics
Visual Studio Code	Writing and running project code

3.2 Project Workflow

1. Load the California Housing dataset using scikit-learn.
2. Separate the eight input features from the target variable MedHouseVal.
3. Split the dataset into training and testing sets using an 80:20 ratio and random_state=42.
4. Create a pipeline containing StandardScaler followed by Linear Regression.
5. Train the model on the training data.
6. Predict house values for the test set.
7. Calculate MAE, RMSE and R^2 on the test predictions.
8. Save evaluation metrics and actual-versus-predicted values to CSV files.
9. Create a plot to inspect the model's predictions.

3.3 Model Training

The implementation follows a sequence of loading, separating, splitting, preprocessing, training, prediction, evaluation and saving results. The scaler is included in the pipeline so that it is fitted on training data and applied consistently to the test data. The trained Linear Regression model then produces predictions for the held-out test records.

3.4 Project Files

File	Responsibility
linear_regression_house_value.py	Loads data, trains the model, evaluates it and saves results
requirements.txt	Lists the required Python libraries
README.md	Project description and run instructions
linear_regression_metrics.csv	Output containing MAE, RMSE and R^2
linear_regression_predictions.csv	Output containing actual and predicted test values
linear_regression_plot.png	Output plot for actual versus predicted values

3.5 Running the Project

10. Open the project folder in Visual Studio Code.
11. Open the terminal and install the libraries using: python -m pip install pandas matplotlib scikit-learn
12. Run the Python file using: python linear_regression_house_value.py
13. Review the metrics in the terminal and open the generated output files.

An internet connection is required the first time the dataset is downloaded. Runtime depends on the computer and installed library versions.

3.6 Testing of the Pipeline

The pipeline should be checked end to end to confirm that the dataset loads, the model trains without errors, the evaluation metrics are calculated using the test set and the output files are created. The actual-versus-predicted plot can be inspected to see whether predictions broadly follow the ideal diagonal pattern. The numerical results in this report are retained from the supplied original report and have not been independently rerun for this revised version.

CHAPTER 4

RESULTS AND DISCUSSION

4.1 Overview of Model Evaluation

The Linear Regression model is evaluated on the test set of 4,128 districts that were not used to fit the model. MAE and RMSE measure prediction error, while R^2 indicates how much variation in house value is explained by the model. Error values are reported in units of $100,000.

4.2 Model Performance

Model	MAE (× $100,000)	RMSE (× $100,000)	R^2
Linear Regression	0.534	0.746	0.576

The reported MAE of 0.534 is equivalent to an average absolute error of approximately $53,400. The RMSE of 0.746 is equivalent to approximately $74,600 in the target units. The R^2 score of 0.576 indicates that the model explains about 57.6% of the variation in the test-set target under this evaluation.

4.3 Interpretation of Linear Regression Coefficients

Feature	Reported standardised coefficient	Reported interpretation
MedInc	+0.854	Strong increase
Latitude	−0.897	Decrease towards the north
Longitude	−0.870	Decrease towards the east
AveBedrms	+0.339	Increase
AveRooms	−0.294	Decrease
HouseAge	+0.123	Small increase
AveOccup	−0.041	Very small decrease
Population	−0.002	Almost no effect

The coefficients above are reproduced from the original project report. Median income has a positive coefficient, indicating that higher income is associated with higher predicted house value when other features are held constant. Latitude and longitude also have large coefficients, reflecting geographic patterns in the data. Coefficients should be interpreted cautiously because features can be related to one another and a straight-line model cannot fully represent complex regional patterns.

4.4 Challenges and Limitations of the Data

- The target is capped at $500,000, so some expensive districts may have true values above the recorded value.
- The dataset comes from the 1990 United States census and covers California only.
- Each record represents a district, not an individual house.

- Some features contain extreme values, such as unusually high average occupancy.
- The data does not include every potentially useful factor, such as property condition, exact floor area, distance to the coast or school quality.
- Only one train-test split is reported, so performance may vary with a different split.
- Linear Regression may not capture non-linear relationships between features.

4.5 Summary of Results

The supplied results show that Linear Regression provides a useful and interpretable baseline for house value prediction. Its reported test metrics are MAE 0.534, RMSE 0.746 and R^2 0.576. These metrics should be understood in the context of the historical California dataset and its capped target values. They are not evidence of performance on current property prices or other geographic regions.

CHAPTER 5

CONCLUSION AND FUTURE SCOPE

5.1 Conclusion

This project demonstrates a supervised machine-learning workflow for predicting a numerical target using Linear Regression. The workflow includes loading a documented dataset, exploring the data, preprocessing features, splitting the records, training the model, evaluating predictions and saving outputs. The reported evaluation results are MAE 0.534, RMSE 0.746 and R^2 0.576. Linear Regression is useful as a baseline because it is simple and its coefficients can be inspected, but it may not capture complex relationships in housing data.

5.2 Limitations

- The dataset is historical and covers California only.
- The target value is capped at $500,000, which affects the most expensive districts.
- The model predicts a district's median value, not the price of an individual house.
- Only one train-test split is reported.
- The model's predictions are educational estimates and should not be used as professional property valuations.

5.3 Future Scope

5.3.1 Cross-Validation

K-fold cross-validation could be used to estimate performance more robustly than relying on a single train-test split.

5.3.2 Feature Engineering

Additional reliable features, such as distance to the coast or to the nearest large city, may improve prediction quality.

5.3.3 India-Focused Dataset

A future version could use a reliable, appropriately dated Indian property dataset. The data source, geography, collection date and price units should be documented clearly.

5.3.4 Web Application

A simple web interface built with Streamlit could allow users to enter the input features and see the predicted district-level value.

5.4 Final Remarks

The project shows how a machine-learning model can be trained and evaluated using structured data. It also highlights the importance of stating the dataset's source, age, target definition and limitations alongside the results.

CHAPTER

REFERENCES

14. Pace, R. K. and Barry, R. (1997). Sparse Spatial Autoregressions. Statistics and Probability Letters, 33(3), 291–297.
15. Scikit-learn Developers. fetch_california_housing documentation: https://scikit-learn.org/stable/modules/generated/sklearn.datasets.fetch_california_housing.html
16. Scikit-learn Developers. LinearRegression documentation: https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LinearRegression.html
17. Scikit-learn Developers. StandardScaler documentation: https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html
18. Scikit-learn Developers. Model evaluation documentation: https://scikit-learn.org/stable/modules/model_evaluation.html
19. Pandas Developers. pandas documentation: https://pandas.pydata.org/docs/
20. Hunter, J. D. (2007). Matplotlib: A 2D Graphics Environment. Computing in Science and Engineering, 9(3), 90–95.

APPENDIX A: PYTHON CODE AND SCREENSHOTS

This appendix adds the runnable Linear Regression code and visual evidence. The code downloads the California Housing dataset on first use, so an internet connection may be required.

A.1 Complete Python Code

The complete code is also provided separately as **house_value_linear_regression.py**. Install the required libraries and run it from the VS Code terminal.

```
# House Value Prediction Using Linear Regression
# Dataset: California Housing (1990 census)
# Install: python -m pip install scikit-learn pandas matplotlib


import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score


# 1. Load dataset
housing = fetch_california_housing(as_frame=True)
df = housing.frame
print("Dataset shape:", df.shape)
print("\\nFirst five rows:")
print(df.head())


# 2. Separate input features (X) and target (y)
X = df.drop(columns=["MedHouseVal"])
y = df["MedHouseVal"]


# 3. Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42
)


# 4. Standardize features using training data only
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)


# 5. Train Linear Regression model
model = LinearRegression()
model.fit(X_train_scaled, y_train)


# 6. Predict house values for the test data
y_pred = model.predict(X_test_scaled)


# 7. Evaluate model
mae = mean_absolute_error(y_test, y_pred)
rmse = mean_squared_error(y_test, y_pred) ** 0.5
r2 = r2_score(y_test, y_pred)


print("\\nLinear Regression Results")
print(f"MAE : {mae:.3f}")
print(f"RMSE: {rmse:.3f}")
print(f"R-squared: {r2:.3f}")
```

```
# 8. Save sample predictions
results = pd.DataFrame({
    "Actual_Value": y_test.reset_index(drop=True),
    "Predicted_Value": pd.Series(y_pred)
})
results.head(20).to_csv("house_value_predictions.csv", index=False)
print("\\nSaved first 20 test predictions to house_value_predictions.csv")


# 9. Plot actual versus predicted values
plt.figure(figsize=(7, 5))
plt.scatter(y_test, y_pred, alpha=0.35)
plt.xlabel("Actual median house value (units of $100,000)")
plt.ylabel("Predicted median house value (units of $100,000)")
plt.title("Actual vs Predicted House Values - Linear Regression")
plt.tight_layout()
plt.savefig("actual_vs_predicted.png", dpi=150)
plt.show()
```

A.2 Code Screenshot — Part 1

VS Code-style view — data loading, preprocessing and model training

```
01  # House Value Prediction Using Linear Regression
02  # Dataset: California Housing (1990 census)
03  # Install: python -m pip install scikit-learn pandas matplotlib
04
05  import pandas as pd
06  import matplotlib.pyplot as plt
07  from sklearn.datasets import fetch_california_housing
08  from sklearn.model_selection import train_test_split
09  from sklearn.preprocessing import StandardScaler
10  from sklearn.linear_model import LinearRegression
11  from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
12
13  # 1. Load dataset
14  housing = fetch_california_housing(as_frame=True)
15  df = housing.frame
16  print("Dataset shape:", df.shape)
17  print("\\nFirst five rows:")
18  print(df.head())
19
20  # 2. Separate input features (X) and target (y)
21  X = df.drop(columns=["MedHouseVal"])
22  y = df["MedHouseVal"]
23
24  # 3. Split data into training and testing sets
25  X_train, X_test, y_train, y_test = train_test_split(
26      X, y, test_size=0.20, random_state=42
27  )
```

Figure A1. Code view showing dataset loading, feature/target separation, train-test split, scaling and model training.

A.3 Code Screenshot — Part 2

VS Code-style view — evaluation, predictions and plot

```
01
02  # 4. Standardize features using training data only
03  scaler = StandardScaler()
04  X_train_scaled = scaler.fit_transform(X_train)
05  X_test_scaled = scaler.transform(X_test)
06
07  # 5. Train Linear Regression model
08  model = LinearRegression()
09  model.fit(X_train_scaled, y_train)
10
11  # 6. Predict house values for the test data
12  y_pred = model.predict(X_test_scaled)
13
14  # 7. Evaluate model
15  mae = mean_absolute_error(y_test, y_pred)
16  rmse = mean_squared_error(y_test, y_pred) ** 0.5
17  r2 = r2_score(y_test, y_pred)
18
19  print("\\nLinear Regression Results")
20  print(f"MAE : {mae:.3f}")
21  print(f"RMSE: {rmse:.3f}")
22  print(f"R-squared: {r2:.3f}")
23
24  # 8. Save sample predictions
25  results = pd.DataFrame({
26      "Actual_Value": y_test.reset_index(drop=True),
27      "Predicted_Value": pd.Series(y_pred)
28  })
29  results.head(20).to_csv("house_value_predictions.csv", index=False)
30  print("\\nSaved first 20 test predictions to house_value_predictions.csv")
31
32  # 9. Plot actual versus predicted values
33  plt.figure(figsize=(7, 5))
34  plt.scatter(y_test, y_pred, alpha=0.35)
35  plt.xlabel("Actual median house value (units of $100,000)")
36  plt.ylabel("Predicted median house value (units of $100,000)")
37  plt.title("Actual vs Predicted House Values - Linear Regression")
38  plt.tight_layout()
39  plt.savefig("actual_vs_predicted.png", dpi=150)
40  plt.show()
```

Figure A2. Code view showing evaluation metrics, saving predictions and creating the actual-versus-predicted plot.

A.4 Sample Terminal Output

Terminal — sample output format

```
Illustrative terminal output — replace with a screenshot from your own run
Dataset shape: (20640, 9)

Linear Regression Results
MAE : 0.534
RMSE: 0.746
R-squared: 0.576

Saved first 20 test predictions to house_value_predictions.csv

Note: Metrics above are the values recorded in the earlier report;
they have not been independently rerun in this environment.
```

Figure A3. Illustrative terminal output using the metrics previously recorded in the report. These results were not independently rerun in this environment; run the code to capture your own verified output.

A.5 How to Run the Project

21. Install Python and open the project folder in Visual Studio Code.
22. Open Terminal → New Terminal.
23. Run: python -m pip install scikit-learn pandas matplotlib
24. Save the provided code as house_value_linear_regression.py.
25. Run: python house_value_linear_regression.py
26. After successful execution, check the terminal metrics, house_value_predictions.csv, and actual_vs_predicted.png.

Important: The California Housing dataset describes California districts from the 1990 census; it is a learning dataset, not a source for current Indian property prices.